

从复现到改进

让 DatawiseAgent 的任务适配、长交互和资源预算可控

InfiAgentBench

DataModeling

structured harness

resource-aware

DatawiseAgent 复现与改进小组

2026-06-09

Takeaway: 复现之后，我们不把问题简化为“多加 prompt”，而是把数据分析 agent 改造成可路由、可验证、可预算的执行过程。

复现以后，三条值得优化的线

方向	核心问题	当前材料状态
1. 任务适配	不同 benchmark 的失败模式不同，不能靠一套统一 prompt 解决。	InfiAgentBench 已有 answer-aware 证据；DataModeling 已推进到 quality-backed harness 与代表性复测。
2. 结构化时间换性能	长交互必须变成可验证、可停止、可修复的过程。	small set light/full 结果完整；DataModeling 的 manifest / quality gate 暴露了 completion 与 performance 的断点。
3. 资源感知调度	强模型、小模型、确定性模块如何分工，减少等效 token 和无效对话。	已有 EqTok 方案和风险调度口径；还缺 token meter 与 E0-E3 全量验证。

Takeaway: 三条线不是堆模块：先识别任务错法，再把交互结构化，最后按风险决定模型、记忆和预算。

证据等级：哪些是结果，哪些是计划

Observed / 已观察

已经跑出来的结果。

- ▶ small set light/full
- ▶ qwen35b DataModeling: 0.5318
- ▶ qwen3-vl: 74/74 completion

Diagnosed / 已诊断

从错误案例和日志得到的分析。

- ▶ InfiAgentBench 口径/格式错
- ▶ DataModeling 低信息提交
- ▶ hard split 重 harness 退化

Planned / 待验证

下一轮需要验证。

- ▶ manifest-only repair
- ▶ 可控遗忘 memory
- ▶ EqTok / E0-E3 消融

Takeaway: 我们区分 observed、diagnosed 和 expected: 结果页给数值, 方案页讲为什么下一步有收益。

Part I

Task-Specific Adaptation

不同任务错法不同，harness 应该跟着任务走

Part I Baseline: 两个 benchmark 的错法不一样

InfiAgentBench baseline / strong reference

Setting	ABQ	PASQ	UASQ
Qwen3-32B full latest	81.74%	86.21%	86.43%
Qwen3.5-35B-A3B baseline	86.38%	90.16%	90.35%

DataModeling baseline / strong reference

Run	Complete	Perf.	Avg time
qwen25	68/74	0.4290	760.25s
qwen35b-a3b-full	73/74	0.5318	288.60s

共同结论

强模型参考有意义：它告诉我们上界、失败口径和“少走弯路可能更快”。但不同任务的主要错误源不一样。

Takeaway: A3B 在这里作为 baseline / strong reference 保留；它不进入第三部分的资源调度候选。

Part I Analysis: InfiAgentBench 更像答案口径问题

常见失败

- ▶ 最终答案格式、小数位、引号、dict/list 格式错误；
- ▶ 预处理顺序、子集口径、缺失值处理错误；
- ▶ 相关性、显著性、正态性、异常值判断错误；
- ▶ 特征工程公式理解错误；
- ▶ reformat 从长日志中漏抽、错抽。

关键判断

很多题不是“代码完全不会写”，而是：

- ▶ 题目约束没有提前固定；
- ▶ 统计语义没有过程校验；
- ▶ 最终答案通道不稳定。

Takeaway: InfiAgentBench 需要 answer-aware harness：盯答案名、口径、统计方法和最终格式。

Part I Scheme: InfiAgentBench answer-aware harness

模块	作用	对应错误
DataCard	固定 CSV 结构, 记录列名、dtype、缺失值、范围、类别样例。	列选择、缺失值、类型误判
Contract	抽取 answer name、列、过滤条件、操作、小数位、输出容器。	口径、格式、额外清洗
Skill checklist	针对 correlation、outlier、ML、format 等题型给检查清单。	统计方法、ML 协议
Verifier	检查列、样本数、p-value、abs(r)、split、final block。	代码能跑但语义错
Final Block	机器可读答案块, 再确定性转 <code>@answer[value]</code> 。	reformat 漏抽、占位符

Takeaway: 它把原本散在 notebook 长日志里的关键信息拆成结构化产物, 方便检查和复用。

Part I Scheme: DataModeling 从 protocol 到 quality-backed

已实现链路

- ▶ Contract / Profile: 固定 metric、列、行数、target;
- ▶ Task Router: tabular、text、time-series、multi-output;
- ▶ Submission Validator: schema、ID、概率范围、sample copy;
- ▶ Quality Gate: manifest、fallback、常数预测、低信息 final。

现在能识别的断点

- ▶ 结构合法但只是 fallback;
- ▶ 多分类塌缩成单类;
- ▶ 回归/概率输出全局均值;
- ▶ CSV 可评, 但没有 validation 和选择依据。

Takeaway: DataModeling 的 harness 不是替模型建模, 而是让 submission 从“能交”变成“可检查、可拒绝、可修复”。

Part I Result: DataModeling 的结果和边界

Setting	Complete	Perf.	Avg time
qwen25 baseline	68/74	0.4290	760.25s
qwen35b-a3b-full	73/74	0.5318	288.60s
qwen3-vl-8b old harness	74/74	0.3127	799.09s
qwen3-vl-32b old harness	74/74	0.2873	297.49s

+0.1028
qwen35b vs qwen25

74/74
qwen3-vl completion

0.45–0.50
修复典型模式后的目标区间

qwen3-vl 的问题不是不能交文件，而是合法 CSV 里常有均值、常数、fallback 或 manifest 缺失。

Takeaway: DataModeling 已经证明强模型与工程约束有收益；弱模型暴露的是 completion 到 performance 的关键断点。

Part I Insight: DataModeling 不是单一难度任务

有逻辑可循的任务

Titanic、Santander 这类 schema / metric 清晰的 tabular 任务，contract 与 validator 能帮助模型走到有效路径。

低分集中模式

多分类塌缩、均值回归、文本错路由、多输出常数、时间/分组任务错当普通表格。

下一步抓手

manifest-only repair、分级 early-stop、任务族模板、低信息预测拦截。

代表复测	分数	说明
8B Titanic	0.7877	有效 tabular submission; manifest 需要短修复
32B Titanic	0.7709	大模型不自动更好，仍受流程厚度影响
32B Santander	0.8116	预测质量可用，耗时和 manifest 是主要瓶颈

Takeaway: 弱模型低分帮助定位：哪些题需轻约束，哪些题必须进入质量驱动流程。

Part II

Structured Time-for-Performance

多花时间不是多问几轮，而是把时间变成可验证过程

Part II Baseline: 原作现象与复现后的问题

原作里有用的现象

弱模型可以靠 notebook-style 长交互补一部分能力：

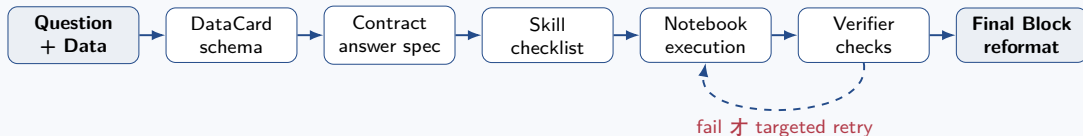
- ▶ planning;
- ▶ code execution;
- ▶ debugging;
- ▶ repair 后继续推进。

复现后看到的问题

- ▶ 长交互不一定更好;
- ▶ full harness 在 hard split 上可能退化;
- ▶ 多分支搜索可能选错;
- ▶ verifier 可能过度阻断;
- ▶ final block 混进 code cell 会卡住 debug loop。

Takeaway: “时间换性能” 不能理解成无脑多跑几轮，关键是多出来的时间是否有明确检查目标。

Part II Scheme: 把长交互拆成可检查环节



执行前： 固定数据结构、答案契约、禁止动作。

执行后： 只在 verifier fail 后短上下文修复，不默认 full 多分支。

Takeaway: 这条线的核心不是模块数量，而是每一轮交互都有输入、检查和停止条件。

Part II Result: small_error_set 先说明趋势

Setting	ABQ	Hard ABQ	Avg runtime
baseline + original reformat	48.94%	44.00%	30.24s
harness_light	59.57%	60.00%	47.03s
harness_full	63.83%	64.00%	92.79s

+10.63

light vs baseline ABQ

+16.00

light vs baseline Hard ABQ

+16.79s

light runtime 增量

保守边界

这是 47 道 known-error small set, 不是 full 257 结论。

Takeaway: light 提升明显且时间可控; full 分数更高但太慢, 说明 full-everywhere 边际收益低。

Part II Analysis: 强模型参考提醒我们别说过头

Setting	Split	ABQ	PASQ	UASQ
Qwen2.5-72B paper	full	81.71%	87.27%	85.09%
Qwen3.5-35B-A3B baseline	full	86.38%	90.16%	90.35%
Qwen3.5-35B-A3B baseline	medium	88.51%	91.71%	91.45%
Harness rerender	medium	91.95%	93.06%	93.42%
Qwen3.5-35B-A3B baseline	hard	81.82%	88.54%	89.16%
Harness rerender	hard	78.41%	84.75%	87.19%

能讲的：medium 上 harness 可能有帮助。

不能乱讲的：hard 上更重的 harness 可能退化。

Takeaway: hard 题不是“多想就好”，还要能选对分支、复算口径。

Part II Case: task 733 卡死在哪里

原问题

FinalAnswerBlock 的 JSON 被写进 Python code cell。

- ▶ JSON 里的 false/null 进入 Python;
- ▶ 触发 NameError;
- ▶ debug loop 反复修, 整题卡住。

修复后

- ▶ final block 只进 markdown / metadata / reformat 输入;
- ▶ code-cell guard 检查 true/false/null;
- ▶ 加 request timeout 和 continue-on-error。

复跑: harness_light success, harness_full success。

Takeaway: 这类问题不是模型能力问题, 是答案通道和执行通道混在了一起。

Part II Thinking: 时间要花对地方

light-first

默认只加最关键的结构化约束和 finalizer，让大多数题先稳定完成。

targeted retry

只有 verifier fail 或高风险题，才追加修复、搜索或升级模型。

hard-aware

hard 题不能只加分支数量，还要能做分支选择和语义复算。

Takeaway: 原作的长交互机制可以继续推进，但要从“交互轮数”变成“可验证的执行预算”。

Part III

Resource-Aware Scheduling

既然知道时间该花在哪里，就可以决定由谁来花、是否要花

Part III Baseline: 用 EqTok 重设资源指标

等效 token 口径

$$\text{EqTok} = T_{32B} + 0.5T_{14B} + 0.25T_{8B}$$

32B 的 token 算 1 个, 14B 算 0.5 个, 8B 算 0.25 个。

32B baseline 指标	数值
ABQ	81.74%
PASQ	86.21%
UASQ	86.43%
调用次数	约 1,621
EqTok	5.961M

Takeaway: 这一部分只讨论 32B / 14B / 8B 的资源方案, 避免把强模型参考和资源路由候选混在一起。

Part III Scheme A: 小模型使用规则

阶段	默认资源	使用原则
Schema / Contract / Verifier / Finalizer	deterministic 优先	能程序化就不问模型；同一 CSV 的 schema 和列映射复用。
Easy planning / routing	8B / 14B / template	短上下文、低风险、格式化强的子任务交给小模型或模板。
Medium planning/debug	14B	用契约固定口径，verifier fail 后只修失败点。
Medium/hard code generation	32B	代码生成和复杂统计口径仍保持强模型主导。
Hard reasoning / fallback	32B thinking	只有高风险或多次校验失败才启用。

Takeaway: 模型路由不是为了全程用小模型，而是把强模型留给真的需要强推理的阶段。

Part III Scheme B: 减少模型对话发生

机制	作用	影响
deterministic artifacts	schema、contract、finalizer 尽量程序化，不反复询问模型。	降低调用次数
contract-to-operation path	从答案契约直接生成操作计划，减少自由探索。	降低 token 和偏题
one-shot execution	低风险题一次模型调用完成。	降低时间
verifier-gated retry	失败才短上下文修复，不把完整日志再喂回去。	控制 retry 成本
artifact reuse	同一数据文件复用 schema 和列名规范化。	降低重复探索

Takeaway: 这比狭义 solver 更 general: 目标不是覆盖固定统计公式，而是减少不必要的模型对话。

Part III Expected Result: 资源消融怎么验证

方案	ABQ	EqTok
32B baseline	81.74%	5.961M
小模型规则	80.5–82.5%	约 2.120M
小模型规则 + 对话减少	81.0–83.5%	1.05–1.60M

后两行都是 expected / target，尚未真实验证。

实验	设置
E0	32B baseline
E1	model routing only
E2	conversation reduction only
E3	full resource-aware

每组报告 ABQ/PASQ/UASQ、调用次数、分模型 token、EqTok、runtime、错误类型。

Takeaway: 资源优化要在题目完成指标不过分下降、调用次数不过分增加的前提下最大化 EqTok 降幅。

Part III Thinking: 资源调度要接住前两部分

强模型不一定更慢

强模型可能减少 debug / repair 轮数，所以总时间可能接近甚至更短。

小模型不一定更划算

小模型只有在简单、明确、短上下文子任务上才适合。

核心是风险调度

low risk 用 deterministic / small instruct; high risk 才用 32B thinking。

Takeaway: 资源调度不是独立省资源技巧，而是任务适配和结构化交互之后自然出现的第三步。

Failure Insights: 负结果告诉我们怎么设计系统

1. Harness 要适配模型

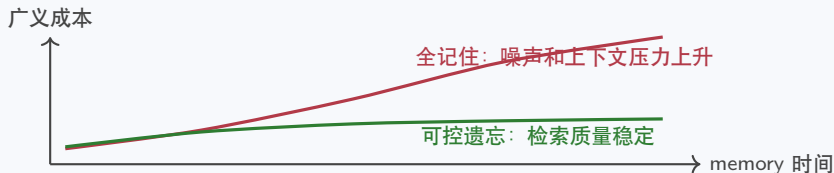
弱模型不能配过厚、过死的流程；目标是按风险调度，而不是 full-everywhere。

2. 记忆不能全记住

旧记忆、低价值记忆会降低信噪比；记忆系统需要重要性、时效性和遗忘机制。

3. Training-free 有上限

DataModeling 跨任务难度大；工程能改路径和验证，但不能完全替代强模型或后训练。



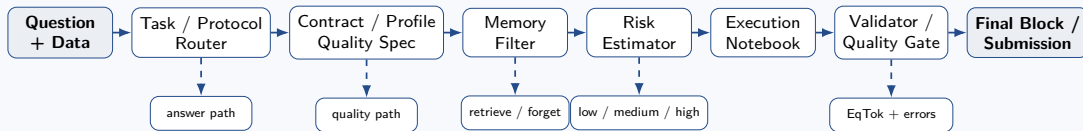
Takeaway: 失败 insight 把边界讲清楚：模型能力、记忆质量和 harness 厚度必须一起调。

Final Integration

Resource-Aware DatawiseAgent

从三个局部改进到一个统一系统

融合架构：任务、时间、记忆和资源在同一条链上



Takeaway: 任务决定检查项，记忆决定上下文质量，验证决定是否继续，风险决定由谁来做。

融合后的四个发现

1. 失败模式不是统一的

InfiAgentBench 错在答案口径和统计协议；
DataModeling 错在 submission、metric、
manifest 和低信息预测。

2. Completion $\neq$ Performance

qwen3-vl 能做到 74/74 结构完成，但
performance 低，说明合法 CSV 还必须经过
quality gate。

3. 记忆和 harness 都要有选择

全记住会降低检索信噪比；full harness 也会
让弱模型负担过重。关键是保留高价值上下文。

4. 最终系统应该做风险调度

low risk 用 deterministic / small instruct,
medium risk 用 small model + verifier, high
risk 用 strong model + targeted fallback。

Takeaway: 我们不是只追求“跑通”，而是判断：任务是否适配、过程是否可验、结果是否有信息量、资源是否值得。

当前差距：下一步要补什么

目标项	当前状态	证据	下一步
weak full 257 baseline vs light	未完成	只有 47 题 small set	跑 full 257 baseline / harness_light
DataModeling harness	主体已实现	contract / profile / router / quality gate	quality repair + case study
qwen3-vl quality gap	已定位	74/74 completion; perf 0.3127 / 0.2873	修低信息预测与 manifest
memory / forgetting token meter / EqTok model routing	方案待验证 未完成 待验证	失败 insight 明确 只有 runtime, 缺分模型 token 目前只有 light/full 两档	加重要性评分和可控遗忘 加 token meter 做 E1 / E3 smoke

Takeaway: 当前缺的不是“有没有 harness”，而是 quality repair、记忆遗忘和资源计量的闭环验证。

DatawiseAgent 的改进不是一次性 prompt,
**而是一套能暴露失败、约束过程、
复用经验、按风险调度模型的
agent 工程框架。**

任务决定检查项

记忆决定上下文

验证决定是否继续